Sofron Vasile-Stelian

FULL-STACK DEVELOPER · BACKEND FOCUS

Bacău, Romania · open to Iași or remote
0722 566 100 · sofron_vasile123@yahoo.ro
linkedin.com/in/vasilestelian ·github.com/VasileStelian

SUMMARY

Full-stack developer, mostly backend. I work in PHP and Laravel with PostgreSQL and MySQL, and I have taken a product from design to production on my own: data model, business logic, REST API, security and deployment. I also handle the frontend when a feature needs it. I work spec-first and test-driven, and I am used to having my code reviewed, including on security. My background is in IT engineering, so databases, Linux and infrastructure are familiar territory, and moving from operations into building products is a good measure of how quickly I pick things up. I would rather ask a question early than commit something I do not fully understand.

TECHNICAL SKILLS

BACKEND	PHP 8.3, Laravel, Filament, REST API design, queue workers, schedulers, service and Action-class architecture, business logic separated from controllers
DATABASES	PostgreSQL, MySQL, schema and migration design, indexing and constraint design (unique and partial-unique indexes), transactional integrity, query performance
TESTING & PROCESS	Pest, test-driven development, unit and integration testing against real databases, code review including security review, spec-then-build workflow
WEB SECURITY	OTP and email verification, MFA (TOTP), CSRF protection, rate limiting, CORS, brute-force lockout, secure setup flows, OWASP-style fixes
DEVOPS	Docker (FrankenPHP), Coolify, VPS deployment (Hetzner), Cloudflare (R2, Pages, Workers), GitHub Actions CI/CD, Git, Linux administration
ALSO	TypeScript, React, Next.js, Astro, Node.js, Python; DNS and email infrastructure; monitoring and incident response

PROJECTS

Apollo Booking

IN PRODUCTION

Appointment platform, my own product

PHP 8.3 · Laravel · Filament · PostgreSQL · Pest · Docker/FrankenPHP · Coolify · Cloudflare

CONCURRENCY	Made double-booking structurally impossible rather than merely unlikely, by moving the rule into a partial unique index in PostgreSQL. An application-level check leaves a race window — two requests can both read "slot is free" before either writes. A database constraint is atomic, so the guarantee holds however many workers run. The cost, accepted deliberately: the error arrives as a constraint violation the application has to translate into something a person understands.
RESKIN, NOT FORK	Built so that a second client is a marketing skin rather than a branch of the product. The Laravel product is client-agnostic; each client gets its own static site pulling services, specialists and gallery over a read-only JSON API with rate limiting and CORS. Onboarding a client never touches the code that runs the bookings.
ONE CODE PATH	Isolated business logic in Action classes so the same code serves both the admin panel and the public API, and can be tested without an HTTP request. 300+ Pest tests run on every change, and each booking rule is verified once instead of twice.
DEPLOYS ITSELF	Editing a service in the admin rebuilds the marketing site with no manual step: Laravel observers fire deploy hooks on content change and Cloudflare Pages rebuilds against the API. Publishing used to mean editing in one place and remembering to redeploy the other.
BOOKING INTEGRITY	Closed the two paths that matter on a public booking form. A booking cannot be confirmed by an email address the requester does not control, because confirmation requires an emailed OTP; and the first-run setup wizard cannot be claimed by whoever reaches the URL first. Admin accounts sit behind TOTP.
RUNS ON MY PAGER	Dockerised on FrankenPHP and deployed to a Hetzner VPS through Coolify with a queue worker and scheduler, media on Cloudflare R2. When it breaks at 2am it is mine to fix, which is a different relationship with your own code than shipping it over a wall.

apollobarbershopacademy.ro · booking.apollobarbershopacademy.ro

FINFORGE

IN DAILY USE

Self-hosted household budgeting app

React 19 · TypeScript · Vite · Fastify · SQLite · Drizzle · Docker

NO LOST EDITS	Two people editing the same budget on two phones cannot silently overwrite each other. Every save carries a version and applies in one transaction scoped to what the caller may write; a stale version is rejected before anything is written, and other sessions are notified over SSE and re-fetch. The failure this removes is the quiet one — an edit that disappears and nobody ever learns it happened. The cost: version-per-save is coarse and rejects some edits that would not truly have conflicted. Invisible at two users; at scale it would need finer granularity.

BOUNDARY HELD — Moved the app from localStorage to a real backend by changing the adapter and one hook — not the pages. The domain logic (surplus, recurring projection, goal allocation) sits in pure functions with no framework or storage access, and persistence sits behind a single storage interface. The migration was the proof that the boundary was in the right place.

AUTH DONE PROPERLY — argon2id hashing, opaque session tokens in httpOnly cookies, double-submit CSRF, and per-route rate limits with the tightest bucket on login — on a personal project nobody would have audited, because the data is our household finances.

DECISIONS ON RECORD — Architecture decisions are written down as ADRs, including the ones that deliberately keep features out. The record of what was refused is the part that stops a small tool turning into an unmaintained big one.

github.com/VasileStelian/registru

Security work on a Laravel training platform

SHIPPED

sigurantapenet.ro, inherited codebase

Laravel · MySQL · Blade · Tailwind

PASSWORDS MIGRATED — Moved the entire user base off unsalted MD5 without a single forced reset or a minute of downtime. Expiring every password would have cost an email campaign, a support spike, and every dormant account that never reads the email. Instead the login path detects the legacy format, verifies against it once, and writes bcrypt in its place — users noticed nothing. The cost, documented rather than hidden: dormant accounts never sign in, so they never migrate, and the legacy verification path stays alive until a planned cutoff.

FREE TO PAID CLOSED — Closed a path from a free account to a paid business plan. Three password-change endpoints were reachable by any authenticated user and unscoped to the caller, so one of them could be used to reach handlers belonging to other account tiers, including the one that upgrades a plan.

CLOSED BY DEFAULT — Flipped the default from open to closed across 14+ unauthenticated routes that exposed user enumeration and company data export. Rather than adding a check to each controller, enforcement moved into role-based middleware, so a route added a year from now is protected unless someone deliberately opts out. The failure mode changes from "forgot to add a check" to "explicitly removed one".

VULNERABILITIES FIXED — A SQL injection in the video listing query, missing validation on two rating endpoints, and password reset tokens generated with mt_rand() instead of a CSPRNG — predictable enough to take over an account without ever knowing the password. All of it surfaced in a review I started myself, on code I inherited rather than wrote.

37 PAGES MIGRATED — Introduced Tailwind v4 into a three-year-old Bootstrap/SCSS codebase behind an explicit opt-in allow-list, migrating 37 pages incrementally with no regressions to anything untouched. No freeze, no big-bang rewrite, and no page left broken while the migration was half done.

HANDED OVER, NOT PATCHED — Reported every finding to the owner with commits and written explanations, and stopped at the ownership boundary: issues in a module owned by another engineer were documented with reproduction steps and handed over rather than patched unasked. Remediation is delivered and awaiting deployment on their side.

sigurantapenet.ro

PROFESSIONAL EXPERIENCE

IT Systems & Security Engineer

JUNE 2024 — PRESENT

Class IT Outsourcing (Startech Team) · Fully remote

FIVE STEPS TO TWO — Cut a five-step routine down to two by building an internal browser extension in JavaScript (Service Worker, DOM injection). Checking a monitored host previously meant opening the ticket, copying the hostname, switching to the monitoring tool, pasting it into a wildcard search and reading the result. The extension surfaces host status directly inside the ticket and links straight through to the host page. Used by the team on every monitoring ticket.

ROOT CAUSE, NOT RESTART — Traced intermittent mail delivery failures that had resisted guesswork, by correlating Microsoft 365 message-trace data against Linux relay logs. The trail led to a timed-out IP and from there to an outbound firewall block — a cause that no amount of restarting the mail service would have surfaced.

WORK THAT RUNS ITSELF — Replaced recurring manual operational work with PowerShell tooling: scheduled jobs, notifications and reporting that run whether or not anyone remembers them. The value is not the time saved once, it is that the task stops depending on a person being available.

BACKUPS THAT REPORT — Bash tooling and cron-scheduled jobs on production servers, including incremental file backups with rsync and Borg, with retention policies and failure alerting. A backup that fails silently is not a backup, and the alerting is the part that makes the difference.

50 TO 60 ESTATES — Escalated incidents and cross-system root cause analysis across 50 to 60 client organisations on Windows, Linux (RHEL/CentOS) and cloud, under SLA-driven ticketing: database and mail infrastructure, DNS, and monitoring with Nagios and Wazuh. Seeing that many production estates fail in that many ways is the reason I write backend code the way I do.

Onsite IT Support Technician

SEPTEMBER 2023 — FEBRUARY 2024

BSlash

BEHIND THE APP Onsite troubleshooting across the stack — web applications, Linux servers, FortiGate firewalls, endpoint deployment. Having access to the systems and logs behind an application is where I started tracing problems to their cause instead of escalating them, and what pushed me toward building rather than only fixing.

Service Desk Technician (L0/L1)

SEPTEMBER 2021 — OCTOBER 2022

SCC Services Romania

DIAGNOSING BLIND First-line support over the phone against a ticketing process with defined escalation paths. My first IT role, and where I learned to diagnose from a description alone and write down what I did so the next person could pick it up.

EDUCATION & LANGUAGES

EDUCATION Bachelor's Degree in Information Technology Engineering, Vasile Alecsandri University of Bacău, 2019 to 2023

THESIS PROJECT A ticketing and administration system for student accommodation, built in Laravel 9 with MySQL and Blade, covering authentication, a booking flow and data-table driven management screens

CERTIFICATIONS JavaScript Development, Software Development Academy, 2023

LANGUAGES Romanian (native) · English (professional)